Analyzing soft costs and benefits of enterprise migration to structured authoring environments

Experience Report

Oliver Fischer
North Carolina State University
oafische@ncsu.edu

ABSTRACT

Structured authoring enables the consistent organization of information in documentation via standardized approaches. Not all enterprises need or benefit from structured authoring. The purpose of this project is to examine the soft costs and benefits associated with converting legacy documentation to a structured authoring environment with the goal of providing enterprises with information to help them make informed decisions.

KEYWORDS

Enterprise Content Management, structured authoring, content management, topic-based authoring, DITA, ASD S1000D, XML, ROI

1. INTRODUCTION

As products become increasingly complex, the importance of effective documentation to provide users with relevant information has also grown[1]. Structured authoring is an approach that can help improve organization and management by assigning a hierarchy and metadata to content. In addition to overall efficiency improvements, structured authoring can also help authors process documentation faster[2]. Large organizations often benefit from a structured authoring environment for documentation, but the business case is not always clear for smaller companies[3]. The cost of implementation for software, training and downtime during the transition are major deterrents to organizations considering the migration to a structured authoring environment, although resistance to change can also be a factor[4]. Furthermore, there are different standards to choose from, making the choice difficult for inexperienced organizations. O'Keefe and Pringle suggest that as tools for structured authoring improve, the cost of implementation will decrease. But what are the factors that enterprises should consider when evaluating the migration to structured authoring? What kind of challenges can technical writers and enterprises expect to face in the process?

To help enterprises make an informed decision as they consider moving to a structured authoring environment, I will be conducting an analysis of soft costs and benefits based on the conversion experience of legacy documentation as a technical writer. The legacy documentation will be converted into DITA and ASD S1000D, two major standards in structured authoring. Organizations may find themselves in an comparable situation when ECM (enterprise content management) solutions are marketed toward them with the promise of improved documentation workflow and efficiency. These advantages would be particularly lucrative for established companies with growing needs for management. But the true investment costs, particularly in terms of rhetorical work necessary, may not be sufficiently communicated ahead of time in favor of an easy sale[6]. ECM solutions may include software packages and standards functionally similar to DITA and ASD S1000D. Anderson (2008) is critical of the claims ECM vendors make about their solutions. Businesses aren't receiving guidance from technical communicators, so it can be difficult for companies to determine whether structured authoring is the correct choice for their specific business environment and needs.

AC Schnitzer, a mid-size business that focuses on aftermarket parts for automotive applications, publicly provides legacy documentation for their products and is a suitable choice for this project. The conversion experience will help outline the challenges and costs that AC Schnitzer may face if they were to move to a structured authoring environment. Ideally, the findings from this project would also be applicable to companies in a similar position as AC Schnitzer. Although different standards may be preferable for specific product lines, the experience will answer whether DITA and ASD S1000D, each designed for different purposes, are useful standards for hardware applications such as aftermarket vehicle parts.

2. LITERATURE REVIEW

At the heart of structured authoring lies the concept of content management (CM), sometimes also called topic-based information development. Minimalism, as described by Carroll (1998), plays an integral role too. CM references specific publication management practices and is often topic-based. The topics are created following predefined standards to ensure consistency and reusability while keeping a minimalist approach in mind to reduce obstacles for the learner. The final product can be published in different formats to best suit the audience, be it a HTML website, eBook, classic PDF, or various other media[5].

Enterprises can choose from several predefined standards, systems and frameworks for content management that have evolved over the years. Technical writers must choose a system if they wish to create modular content, although enterprises with sufficient resources may choose to develop their own from scratch[1]. Most systems rely on XML (Extensible Markup Language). XML documents contain both content (such as textual information) and information about the structure of the document. XML tags can be encoded by other XML tags, creating a hierarchical structure of information. Individual elements within the XML document can be retrieved independently from the document itself[8], making the process content based. XML serves as the backbone of many popular structured authoring systems.

While XML provides the structure within documents, the rules for these structures are provided by a DTD (Document Type Definition) or schemas, where XML rules are defined. These elements are what structured authoring builds on. Typical XML elements may include tags defining paragraphs or safety warnings, which can create both structural and semantic meaning. In addition to these elements, a DTD also defines rules about how to use these elements. For example, a DTD may define that a safety warning must include, in this order, a signal to draw attention to the warning, the type of danger that the reader may face and a description of the danger. This standardization is what creates consistency and efficiency across documentation that adheres to structured authoring, both during the writing and review process[1].

By separating content and form, XML promises easy reuse of content, saving both time and money. Searching and retrieving information also becomes more efficient and accurate. Since XML is standardized, it is easy to exchange information with other authors. But keeping its features in mind, XML also foreshadows issues when scaled to an enterprise level.

Expensive implementation costs and maintenance are major deterrents. A specialized DTD can take years to develop and require purposefully developed tools to make use of a custom markup language. In certain industries like software, information categories are constantly shifting, so big upfront investments are risky. Creating a custom markup language means you can tailor it to your content needs, but using custom tags means foregoing interchangeability with external authors. Conversely, adopting a standard markup language means missing out on the benefits of content-specific markup[9].

Although enterprises may create their own structures on the open basis of XML, it is an extremely time, effort and cost-prohibitive option. As such, several standards have emerged that enterprises may choose from. Kothes (2011), provides an overview of the most prevalent systems currently in use. The table below has been updated to reflect the situation as of 2022 (see table 1).

3. METHODS

One of the main objectives of this project is to fill the gap between enterprise level conversations about ECM solutions and technical communicators who are left out of these conversations[6]. By providing enterprises that are considering the migration to a structured authoring environment with the necessary information from the perspective of a technical communicator, I intend to help them make a more informed decision. Several types of cost factors are involved in the process, with labor and training time being major ones[11]. Therefore, determining how much and what type of labor is required during the migration is a key component. Similarly, the benefits to both the reader and author in terms of accessibility and efficiency must be considered. A more detailed breakdown of factors that will be used to determine soft costs and benefits is available at the end of this section.

To achieve the project goal, I will be converting existing legacy documentation into XML documentation using both the DITA and ASD S1000D standard. The company I chose to borrow the legacy documentation from is AC Schnitzer, a German automotive tuning company that creates aftermarket parts for various car brands. I chose AC Schnitzer because it conformed to the following set of criteria:

3.1 It is a hardware company

The case for structured authoring is clearer in the software than the hardware industry, with DITA being specifically designed for software applications[1]. Focusing on a hardware company will test the flexibility of DITA while examining the usefulness of ASD S1000D outside of military hardware and the aviation industry.

3.2 It is a midsize business

The larger a company, the more documentation is needed and the more resources are available. Migrating to a structured authoring environment may be less of a question but rather a necessity for these enterprises. Similarly, a small company likely won't have the need or resources for employing structured authoring. Gartner defines a midsize enterprise as an organization that has a revenue higher than \$50 million but less than \$1 billion annually[13]. AC Schnitzer is a division of the KOHL group, which has 550 employees and a turnover of about \$200 million, falling on the lower spectrum of the Gartner SMB definition[14].

3.3 It provides public access to legacy documentation

750 instructional manuals are readily available online as PDFs designed for print[15].

3.4 Familiarity with products

I have handled and installed the types of products that AC Schnitzer sells, so product familiarity, which is often necessary when creating documentation, should not be an issue.

Based on the concept of typical case sampling, the documentation I will convert are the fitting instructions of the AC Schnitzer muffler for the newest BMW 3 series. Typical case sampling is designed to highlight what is typical or average about a specific setting, without generalizing[10]. Mufflers are typical components frequently installed by end-users and trained professionals in aftermarket settings. The instructions are available here.

Using Oxygen XML, I will start converting each of the 12 English pages into XML documentation, applying the DITA standard as accurately as possible. Some types of information might not directly fit into the framework of DITA, so I expect to implement workarounds where necessary. This will test the claimed flexibility of DITA[1] and determine how suitable it is for hardware applications. Once I have converted all 12 pages, I will repeat the same process for ASD S1000D, answering parallel questions about its usefulness outside the aviation industry and military. Many of the comparisons drawn will be relative to each other.

Traditionally, a cost-benefit analysis would be calculated using the Return on Investment (ROI) evaluation model. This model compares the monetary costs and benefits of a program and expresses them as a benefit and cost ratio or percentage. The formula to calculate ROI is as follows[12].

$$\text{ROI} = (100\left(\frac{Total\ Benefits - Program\ Costs}{Program\ Costs}\right))$$

But following this model creates issues. Access to company financial data is limited. I won't have access to internal AC Schnitzer records and will need to rely on public information as provided on their website and in their marketing materials.

Furthermore, the ROI model focuses solely on monetary costs and benefits which, while decisive for direct investments, ignore less tangible aspects that are significant in the documentation process. Since this project is not designed to act as a framework, but rather as an information piece to help enterprises understand the factors involved with migrating to structured authoring systems, I will be focusing on soft costs and benefits. As previously mentioned, Anderson (2008) highlights that technical communicators are often left out of these types of conversations, so this project also aims to shed some light on the experiences and challenges authors may face in a documentation conversion scenario (such as a company utilizing ECM solutions), from the perspective of a technical writer.

The 2 soft cost factors I will be monitoring are listed below in order of descending importance.

3.5 Time to convert legacy documentation

I anticipate most of my time to be dedicated to the conversion process. I will use the stopwatch on my computer to record the total time required to convert all 12 pages. Time required to implement the standard, learn new concepts, or look up information will be included in this measurement. Multiplying the final conversion time by 750 should also lead to a more complete training cost number, although each document varies in length so the final number would merely be a rough estimate.

3.6 Flexibility

Number of instances in which the standard framework did not allow direct conversion and a workaround was necessary (quantitative).

The 2 soft benefit factors I will be monitoring are listed below in order of descending importance.

3.7 Benefit in reader experience

To define reader experience more categorically, I will evaluate following subcategories:

Factor	Type
Content word count (DITA) Content word count (ASD S1000)	Quantitative
Ease of use (comments based on experience report)	Qualitative

3.8 Anticipated reduction in documentation management

The flipside of benefit in reader experience, evaluating the advantages of the new documentation from the perspective of authors and managers.

4. RESULTS

4.1 Original documentation

To create a point of reference, it is important to assess the original legacy documentation and the user needs that it appears to address. The legacy documentation is a 24-page bilingual PDF document in the format of installation instructions for the AC Schnitzer silencer for the BMW G20 series chassis. The German and English instructions are split evenly at 12 pages each. The instructions are split into 3 main sections that include general notes, a list of supplied components and the actual fitting instructions. The information is presented in a linear fashion and each step in the installation procedure is accompanied by an image.

The instructions and outlined procedures are complete enough to help users accomplish the task of installing the silencer, but not detailed enough to effectively guide inexperienced users through the entire process. Some requirements are only vaguely stated. For example, the instructions call for an Allen key and wrenches, but they do not specify what sizes. Some prerequisites that would require prior planning from inexperienced users, such as the number of people required to complete the installation, are casually introduced in the middle of the installation procedures rather than appearing as a warning or in a list of requirements at the beginning of the document. For instance, step 8 introduced the need to “ask another person to assist.” It stands to reason that the primary audience for these instructions are mechanics working in a shop environment with access to a variety of tools and additional hands when necessary.

4.2 DITA

4.2.1 Time to convert legacy documentation

The total time required for converting the legacy documentation into DITA (one language) was 4h57min19sec. This includes time spent analyzing the document, planning and design work and writing the documentation using Oxygen XML. Results may differ based on the expertise level of individual authors, so the indicated total time should be viewed as an average for converting 12 pages of AC Schnitzer documentation.

AC Schnitzer does not provide a central list of all documentation, so to estimate an average number of document pages to convert, I sampled typical documentation for the BMW G20 series (same chassis that the legacy documentation used in this project is for) to narrow selections down to a single model line. I then selected the first product on the page from each category that had installation instructions available. This created an average of 17.43 pages for all languages or 8.72 pages per language. Using a single language is more accurate. Any additional language would require less time since it is mostly a matter of translation.

8.72 pages would take about 3.63h to convert. Multiplied by 750, that creates a rough estimate of 2725 hours to convert existing AC Schnitzer documentation to DITA.

4.2.2 Flexibility

There were 5 separate instances in which the content or schema of the original document did not match the schema outlined by DITA. Details on the limitations or flexibility issues I ran into are described in the list below.

4.2.2.1 DTD limitation

Refers to the limitations of individual schemas. DITA provides a limited set of DTDs with specific purposes, which do not always align perfectly with the content being converted. In several instances, I had to use the closest substitute that would suit specific content and provide the XML elements I needed. For example, I used a concept document for general notes and fitting time.

4.2.2.2 XML tag limitation

Refers to the elements or XML tags that are available in each DTD. Every DTD has a unique set of elements available to suit the purpose of the DTD, but because I had to substitute DTDs as explained in 4.2.2.1, certain elements that would have best described specific content were not always available in the chosen DTD. For example, I classified certain content in the original document as a prerequisite, but the <prereq> tag is not available in concept documents. Instead, I had to use a different, more universal tag like <p>, which is less descriptive of the content.

4.2.2.3 Page separation does not translate

Refers to the structure of the legacy documentation being built around the idea of pages like those found in a word document or PDF file. For example, certain designations (like letters to number items) are repeated because they appear on different pages in the legacy documentation (a PDF file) and are therefore not related, allowing them to be reused. Because topic-based structured authoring is not bound by pages, the pre-defined designations do not translate. Copying them would create the appearance of duplicate designations, so I resorted to creating new designations. Images with embedded designations would need to be updated to reflect the new designations.

4.2.2.4 Images in reference document types can't be inline

Refers to images not being able to be inserted in the main body of reference DTDs. A workaround is to create an empty table and insert the image there, though this creates inconsistencies in how images are implemented when viewing the final documentation.

4.2.2.5 Inconsistent legacy schemas

Refers to inconsistencies in how the legacy documentation is designed and structured. Since it is not bound by pre-existing elements or DTDs, some content might not follow the logical structure created by preceding pages. For example, in step 19 of the legacy documentation, a separate installation prerequisite for an optional part (carbon sport tailpipe trim) is introduced. This prerequisite is mixed in with the regular installation steps. This can't explicitly be implemented in the same manner in a task DTD. In this case, I used the <note> element as a substitute. This issue is similar to 4.2.2.2, but the cause is different enough to warrant a separate instance.

4.2.3 Benefit in reader experience

(See table 2).

DITA was able to cut down on the total word count by 295 words, a reduction of about 30%. Simultaneously, breaking the legacy documentation down into individual topics has created a more granular structure and detailed overview of the documentation.

4.2.4 Anticipated reduction in documentation management

There are 11 distinct topics in the ASD S1000D structure compared to 4 in the legacy structure. This means there are 7 more steps during the documentation process where reviews can take place.

4.3 ASD S1000D

4.3.1 Time to convert legacy documentation

The total time required for converting the legacy documentation into ASD S1000D (one language) was 6h54min02sec. This includes time spent downloading ASD S1000D, familiarizing myself with the standard, adding the standard to Oxygen XML, designing and finally writing the documentation. Results may differ based on the expertise level of individual authors, so the indicated total time should be viewed as an average for converting 13 pages of AC Schnitzer documentation. Some planning had already been done in the DITA conversion process, although there was still unique design work required for ASD S1000D.

Using the same averages as previously established, 8.72 pages would take about 5.09h to convert. Multiplied by 750, that creates a rough estimate of 3817.50 hours to convert AC Schnitzer documentation to ASD S1000D. That is a 40% increase in time required over DITA.

4.3.2 Flexibility

There were 3 separate instances in which the content or schema of the original document did not match the schema outlined by ASD S1000D. 2 less than DITA. Details on the limitations or flexibility issues I ran into are described in the list below.

4.3.2.1 XML tag limitation

Refers to the elements or XML tags that are available in each XSD. ASD S1000D uses XSD files to create schemas for each of its data modules. Their role is comparable to DTDs in DITA but with one major difference: They are easily extensible with local tools. Out of the box, certain XML tags were not available in locations where I needed them. For example, IPDs don't allow for images. By editing the corresponding XSD file, it would theoretically be possible to make images work in IPDs due to the extensible nature of data modules. I opted out of customizing data modules as the goal of this project is to evaluate ASD S1000D as delivered by ASD, not create custom solutions.

4.3.2.2 Page separation does not translate

Refers to the structure of the legacy documentation being built around the idea of pages like those found in a word document or PDF file. Carries over from DITA because it is a design issue and thus independent from the specific standard being used. For example, certain designations (like letters to number items) are repeated because they appear on different pages in the legacy documentation (a PDF file) and are therefore not related, allowing them to be reused. Because topic-based structured authoring is not bound by pages, the pre-defined designations do not translate. Copying them would create the appearance of duplicate designations, so I resorted to creating new designations.

4.3.2.3 Inconsistent legacy schemas

Refers to inconsistencies in how the legacy documentation is designed and structured. Also carries over from DITA because the issue is rooted in the legacy documentation, so it likely affects most structured authoring standards, though exceptions are possible. Despite the granular and extensive nature of ASD S1000D, there were still instances in which the legacy schema could not be converted due to inconsistent legacy structure. For example, there is no way to add an attention note into a procedural step. It can only be added at the beginning of the data module as a warning before the main procedure starts. The legacy documentation initially places warnings before the main procedure but also introduces them during procedures in later steps. A workaround is to use the <caption> element, which draws more attention to the line both in code and in the final publication, though it doesn't explicitly describe content as needing attention.

4.3.3 Benefit in reader experience

(See table 3).
ASD S1000D increased the total word count by 245 words, or about 19%. Simultaneously, breaking the legacy documentation down into individual topics has created a more granular structure and detailed overview of the documentation. Integrated separation of product and procedure type creates more accurate topic titles compared to DITA.

4.3.4 Anticipated reduction in documentation management

There are 10 distinct topics in the ASD S1000D structure compared to 4 in the legacy structure. This means there are 6 more steps during the documentation process where reviews can take place.

5. DISCUSSION

5.1 DITA

During the design of this project, one major concern was the ability of DITA to adapt to the schema provided by the legacy documentation. Being a software-focused standard, DITA was already at odds with the AC Schnitzer documentation, which outlines hardware procedures. While I expected to spend most of my time implementing workarounds, the time that flowed into design work and planning far exceeded the effort of adapting the legacy schema to DITA. In retrospect, it would have also been more helpful to distinguish between time used on design and planning versus implementation.

Much like the ECM packages that Anderson (2008) criticized, simply owning or being able to use software is a different matter than being able to effectively utilize what it has to offer. This fact became clear during the DITA conversion process. Simply converting content line by line was not only impossible due to schema differences, it would have also created ineffective documentation that did not make use of the features that DITA has to offer. Design choices must be made at low and high levels, from simple decisions such as determining the specific hazard level for a warning (DITA offers four types of hazard warnings based on the danger and potential consequences) to overarching structural decisions about how to break down procedures and tasks to support content reuse and satisfy audience needs.

The original AC Schnitzer documentation was clearly designed with the mindset of creating a self-contained, linear PDF file. Central to structured authoring is the idea of topic-based authoring. While the legacy documentation treats the installation of the silencer as one unit, DITA provides the opportunity (and arguably necessity) to break up this linear procedure into reusable topics. Familiarity with the product family (automotive exhaust systems) and first-hand experience with installation procedures played instrumental roles in the design decisions I made. I created 6 distinct reusable topics covering the hardware procedures in addition to 5 topics covering additional information about the product. Table 2 compares the original structure in the legacy documentation to the structure (topics) I created with DITA.

This topic-based authoring not only allows reuse, which can save time for the authors and streamline the documentation process, but also makes it easier for readers to skip, find or re-visit steps based on their individual needs and goals. Since DITA allows authors to directly publish a responsive Webhelp site which presents each topic as a card, it is easier for readers to see everything at a glance. They receive a better overview of individual tasks involved, rather than having to jump multiple pages back and forth within a linear document (see appendix 1).

Despite DITA being flexible enough to have accommodated most of the original schema and content of the legacy documentation, there were still several shortcomings in the final product. Dividing the original task into topics as outlined above works well for software applications due to the self-contained nature of individual tasks. For example, a user might want to look up how to change a program setting. Afterwards, the user might want to find out how to update the software. The order in which the user looks up either piece of information is inconsequential. This is not typically the case with hardware procedures where individual topics typically need to follow a specific chronological order. In the AC Schnitzer documentation I converted, I was forced to add numbers to each topic to make it clearer which steps users must follow first. For example, it may not be obvious to inexperienced users that they first need to remove the stock exhaust before preparing the AC Schnitzer silencer for installation. Technical writers faced with this problem should consider whether the audience can deduce the correct procedural order without additional pointers. If not, a numbering system is the easiest workaround for this issue.

Furthermore, outside of general design considerations, some of the content had to be adjusted or rephrased once adapted to DITA because the context can change when it is implemented in the new structure. For instance, notices about required steps are no longer necessary since there is a dedicated tag for that type of content. References to specific steps or numbers should also be replaced with xref links. These types of content cuts are what helped DITA reduce the content word count. Some new content may also need to be created to fill in information where new DITA structures create gaps. Hazard warnings, for example, require specific information to be complete, such as describing the type of hazard, consequences and how to avoid them.

Design is another consideration. I used the standard stylesheet for export, but a proper publication would require AC Schnitzer to develop their own custom stylesheet so that their company information, logo and any other information they wish to be part of the document would automatically be applied whenever they publish new documentation. AC Schnitzer would only need to create the stylesheet once and it would apply to whatever new content they produce.

5.2 ASD S1000D

During the design of this project, ASD S1000D appeared to be more suitable for hardware applications, being a standard developed for aviation and vehicles in general. The concern of adaptation issues was lower than with DITA, although still present. Design work once again dominated the time spent during the conversion process. There were less limitations when adapting the legacy schema to ASD S1000D compared to DITA. For example, there is a far greater number of data modules in ASD S1000D compared to DTDs in DITA, allowing for more granular and accurate control over content adaptation. It was easy to assign content to data modules, with the original types of content from the legacy documentation being represented through front matter, description, illustrated parts list and procedure data modules.

This comprehensive and specialized approach comes at a cost. At over 3500 pages for the ASD

S1000D manual, the learning curve is far steeper compared to DITA. To make matters worse, support for ASD S1000D is limited. The included manual, while comprehensive, acts more like a reference than a learning tool. It is useful for searching up elements and style standards, but not ideal for learning ASD S1000D from scratch. A significant amount of time was dedicated to searching for information, supplemented by scarce online resources where necessary. DITA is considerably more accessible and concise in its documentation.

Another major differentiator between DITA and ASD S1000D is the included style guide. ASD S1000D offers a holistic approach that requires strict adherence to its extensive included style guide. These can be as specific as standardized number groupings for specific zones on an aircraft and as general as naming conventions for file names. For example, XML file names are specified in the manual as needing to follow "TYPE-CONTROLNUMBER.FORMAT," although the provided examples additionally make it obvious that region and language according to ISO 3166 standards must be included as well.

Small discrepancies like that are common in the included manual, so familiarizing yourself with the provided bike example documentation is as essential as reading the manual (see appendix 2). Such a detailed style guide can save time for organizations who might otherwise need to spend resources on developing their own style guide. Despite that, the burden of learning the provided style guide does fall on individual authors in this case. Strict naming conventions take time to learn. They are embedded in the schemas sometimes, so certain elements will not allow you to enter content that deviates from a specific format that it expects.

Although it can be used for hardware in general, it is clear that ASD S1000D was designed with the aviation industry in mind. It is difficult to sift through unnecessary information in what feels like a bloated manual. Many rules for proper practices are not specifically mentioned in the included documentation. For example, images must be declared at the top of the XML document prior to insertion in the main body. I had to deduce this rule and learn it from the included example documentation because this information is not present in the section of the manual that explains how to insert graphics.

Much like DITA, data modules in ASD S1000D act as top-level containers in the form of XSD files (versus DITA's DTD files). Examples in DTDs include concept or reference containers. ASD S1000D has a greater number of and more specific XSD containers. This is where the hardware-focus of ASD S1000D bears advantages over DITA. There is a dedicated illustrated parts data module, which is a much more accurate representation of the function of the parts list included in the legacy documentation that I was not able to accurately categorize using DITA. Even something as simple as a title page is not a category available in DITA that ASD S1000 does offer.

Such granular categorization extends to the number of available elements as well, resulting in the ability to tag content with a level of accuracy far exceeding that of DITA. For example, even paragraph elements come in multiple flavors compared to DITA's only option (<p>). They include the general <para> or more specific <notepara> and <warningandcautionpara>, just to name a few. Of course, the sheer number of elements creates a nightmarish scenario for new authors who may find themselves spending hours searching for elements and their required parent elements. For example, <descrForPart> can only appear within

<itemIdentData>, which in turn can only appear in <partSegment>. Working with ASD S1000D when you are unfamiliar with the vast list of elements and typical hierarchical orders often means spending time backtracking element requirements based on the specific elements you need to tag your content. Additionally, XSD data modules in ASD S1000D are extensible, meaning they can be customized to suit individual needs with relative ease. Even if an element is not available in a specific data module, it can be added after the fact.

The detailed nature of data modules in ASD S1000D also creates content gaps that cannot always be filled with the limited information provided by legacy documentation. For example, the illustrated parts data module requires manufacturer codes for each item, which are not provided in the legacy documentation. Detailed product information is embedded in the metadata of every XML document that comprises an ASD S1000D project.
Since each item is meticulously tagged with elements like part numbers or manufacturer codes, cross-referencing between parts becomes easy, which can be a huge benefit to both the author and reader when several parts are included in a system. It helps them keep an overview of parts and allows them to be more specific about which parts they are referencing. Readers may also maintain a better overview and be able to verify that they are using the correct part based on unique part numbers. It also allows legacy content that denotes relationships to be removed. For example, part E no longer needs to specify that it is only for part D because the lists in ASD S1000D have that relationship built in. Every time part E is referenced, that relationship carries over.

Unfortunately, a major limitation of working with ASD S1000D in Oxygen XML is that publishing is not currently supported. As such, any observations were made under consideration of file structures and the author view (see appendix 3).

6. SUMMARY

6.1 DITA vs. ASD S1000D

The conversion process into DITA and ASD S1000D has highlighted a shared and integral role of rhetorical design work. Using the tools still requires a learning curve, one that is particularly steep with ASD S1000D, but the main challenge lies in creating structured content from unstructured legacy documentation. Both standards begin diverging in their implementation processes, with DITA being considerably more approachable compared to ASD S1000D.

The framework that ASD S1000D provides is massive compared to DITA. Data modules are varied and extensible, while DTDs offer a limited set of functions and features that cannot easily be expanded or customized. Equally massive is the manual and time required to acclimate to ASD S1000D’s seemingly endless features, granular XML tags and customizable options. Still, despite its specialized focus on military and aviation, ASD S1000D is at least as effective for general hardware procedures, be it vehicles (as is the case with the chosen AC Schnitzer documentation) or other hardware. Even simple features like a built-in separation between hardware type and procedure in the title using the <techName> and <infoName> tag eliminates the DITA issue of non-descriptive titles.

DITA was flexible enough to accommodate the specific schemas and content of a hardware installation instruction with relative ease, but this flexibility is somewhat deceiving. It is achieved through streamlined schemas that limit expansion beyond basic hardware documentation. It also suffered from a higher number of flexibility issues. In fairness, DITA was considerably easier to work with, which is also reflected in the lower conversion time. Where enterprises might need to spend more time with DITA is with style. ASD S1000D comes with a comprehensive style guide that addresses both content and structure, which should cut down on time otherwise spent on designing a custom style guide while also reducing inconsistencies. Ultimately, both standards are suitable for hardware documentation, but only ASD S1000D has the potential to create a library of interconnected, specialized hardware documentation with minimal limitations.

6.2 The costs

Time and required labor are the primary cost factors that enterprises must consider. While ECM solutions or individual software and license purchases can cost thousands of dollars, something like Oxygen XML remains a one-time cost that pales in comparison to the labor time and cost for converting legacy documentation. This is a process that cannot be automated due to the nature of structured authoring, which, as outlined in the analysis, requires extensive planning and rhetorical design work to take advantage of its beneficial features. A company like AC Schnitzer can expect to pay an estimated 2725 hours' worth of labor to convert their existing library of legacy documentation into DITA. At an hourly wage of $29, the total cost would be in the $79,025 range. With internal data, AC Schnitzer could estimate time saved on managing and maintaining a library of topic-based documentation, using those values to calculate the return on investment as explained in the methods section.

Another cost that may not be immediately felt by enterprises is the time and effort that technical communicators will need to invest in learning a new standard or implementing workarounds. Depending on the specific situation of each company, restructuring of work processes, new training programs or new hires may be necessary, all of which require both time and money. This is especially the case for technical communicators who may not be familiar or comfortable with topic-based authoring in general. These hidden costs should not be ignored in a ROI calculation. Choosing a standard like DITA or ASD S1000D can also yield massively different results. Labor time for conversion with ASD S1000D was 40% higher than with DITA. On the other hand, once the initial learning curve is overcome, the included comprehensive style guide and advanced features of ASD S1000D might be able to reduce labor and management costs in the long run, especially as the company and its requirements grow over time. ASD S1000D should have more potential for growth than DITA due to its extensibility.

6.2 The benefits

Readers benefit quite clearly from the DITA publication, which presents all its topics on a single web page. Keeping an overview is easy compared to scrolling through a 24-page linear document. Furthermore, jumping between topics requires little effort, creating a more tailored experience based on individual user needs. By catering to a larger audience without sacrificing the experience on either end of the user spectrum, enterprises who would otherwise keep multiple sets of documentation for different audiences could save on management costs by reducing their library. Unfortunately, Oxygen XML currently does not support publication of ASD S1000D projects. But due to the topic-based structure of both standards, the reader and management benefits should be shared between these two standards.

Furthermore, topic-based authoring also means more possible checkpoints for reviews, which should create more efficient work processes and help avoid costly mistakes in the final product that might be difficult to fix after publication. The ability to re-use topics across separate documents also cuts down on labor time during and after conversion or production. Maintenance of documentation becomes more manageable as well when working with concise individual topics rather than bulky and long linear documentation.

7. REFERENCES

[1] Kothes, L. (2011). Grundlagen der Technischen Dokumentation. Springer-Verlag Berlin Heidelberg.

[2] Hall, W. (2001) Maintenance Procedures for a Class of Warships: Structured Authoring and Content Management. *Technical Communication, 48*(2), Society for Technical Communication, 235-247.

[3] O'Keefe, S. and Pringle, A. (2017). Structured authoring and XML. Scriptorium.

[4] Adobe (2011). Migration to Structured Authoring [White paper]. https://www.adobe.com/content/dam/acom/en/products/framemaker/whitepapers/Migration-to-Structured-Authoring-Determining-the-HighestROI-Lowest-TCO-Solution.pdf

[5] Andersen, R. (2014). Toward a more integrated view of technical communication. *Communication Design, 2*, 10-16.

[6] Andersen, R. (2008). The Rhetoric of Enterprise Content Management (ECM): Confronting the Assumptions Driving ECM Adoption and Transforming Technical Communication. *Technical Communication Quarterly 17*(1), 61–87.

[7] Carroll, J. (1998). The Nurnberg Funnel: Designing Minimalist Instruction for Practical Computer Skill. MIT Press, Cambridge.

[8] Szlávik, Z., Tombros, A. and Lalmas, M. (2006). Investigating the use of summarisation for interactive XML retrieval. In *Proceedings of the 2006 ACM symposium on Applied computing* (*SAC '06*). Association for Computing Machinery. 1068-1072.

[9] Priestley, M., Hargis, G. & Carpenter, S. (2001). DITA: An XML-based Technical Documentation Authoring and Publishing Architecture. *Technical Communication, 48*, 352-367.

[10] Palinkas, L., Horwitz, S., Green, C., Wisdom, J., Duan, N., & Hoagwood, K. (2015). Purposeful Sampling for Qualitative Data Collection and Analysis in Mixed Method Implementation Research. *Administration and policy in mental health, 42(5)*, 533–544.

[11] Adobe (2019). Determining the Highest ROI Solution for Technical Publishing Migration from Microsoft Word [White paper]. https://www.adobe.com/content/dam/acom/en/products/framemaker/whitepapers/Enterprise_Migration_to_FrameMaker_11_from_MS_Word_for_Technical_Documentation.pdf

[12] Wegenast, D. (2001). Measuring Return on Investment (ROI). *National Human Services Training Evaluation Symposium*, 4-11.

[13] Gartner (2011). Small And Midsize Business (SMB). https://www.gartner.com/en/information-technology/glossary/smbs-small-and-midsize-businesses

[14] AC Schnitzer (2022). About us. https://www.ac-schnitzer.de/en/about-us.html

[15] AC Schnitzer (2022). The AC Schnitzer principle: from motorsports to the road. https://cms.ac-schnitzer.de/fileadmin/mediapool/images/unternehmen/2017-03_Presentation_complete_new.pdf